

СОВРЕМЕННЫЕ ТЕХНОЛОГИИ РАЗРАБОТКИ WEB- ПРИЛОЖЕНИЙ

ЛЕКЦИЯ 8а

Темы занятия

DOM. Объект window

Объектная модель документа

Получение и навигация по элементам DOM

Иерархия классов и свойства элементов DOM

Атрибуты и DOM-свойства

Добавление и удаление узлов

Работа со стилями

Отображение страницы браузером

Основные этапы:

1. Получение html-документа и связанных ресурсов.
2. Построение *объектной модели* страницы.
3. Рендеринг страницы.

При чём тут JavaScript?

1. Скрипты загружаются как часть страницы или ресурс.
2. Код скриптов может взаимодействовать с *объектной моделью* страницы и *браузером*.

BOM и DOM

Объектная модель браузера (browser object model, BOM) – множество объектов, создаваемых браузером при загрузке страницы.

Частью BOM является *объектная модель документа* (document object model, DOM) – дерево объектов, описывающих собственно html-документ.

Стандартизация DOM

Стандартизацию выполняет W3C:

- ❑ DOM Level 1 (1998)
- ❑ DOM Level 2 (2000) – максимальная поддержка
- ❑ DOM Level 3 (2004) – не всё поддерживается браузерами
- ❑ DOM Level 4 – находится в разработке

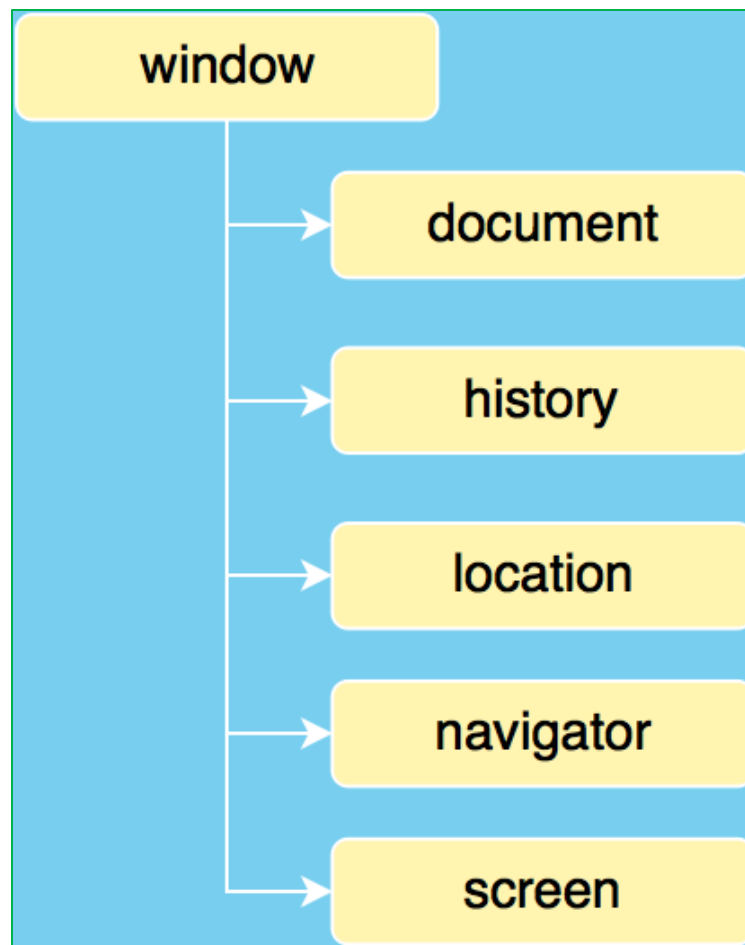
Ещё встречается упоминание DOM Level 0 – это то, что было реализовано в самых первых браузерах.

Объект window

В браузерах **глобальный объект** JavaScript имеет свойство `window`, которое содержит ссылку на сам глобальный объект.

Объект window (для краткости так будем глобальный объект в браузере) рассматривается как главный объект в BOM. Он отвечает за окно браузера, вернее – за вкладку (две вкладки – два объекта window).

Основные свойства window



Основные свойства window

`document` – стартовый объект DOM. Описывает html-документ, загруженный в окно браузера.

`history` – хранит историю посещения страниц в текущем окне. Позволяет перемещаться по истории назад или вперёд.

`location` – отвечает за адресную строку браузера, позволяет получить и изменить адрес. При изменении адреса браузер осуществляет переход по новому адресу.

Основные свойства window

`navigator` – информационный объект с данными самого браузера (название, версия, установленные компоненты).

`screen` – информация обо всём экране (разрешение экрана, глубина цвета).

Основные свойства window

Упомянем другие свойства объекта window:

- `innerWidth`, `innerHeight` – размеры области содержимого
- `frames` – массив фреймов в окне
- `XMLHttpRequest` – объект для асинхронных запросов
- ...

Полное описание смотрим тут:

<https://developer.mozilla.org/en-US/docs/Web/API/Window>

Основные свойства window

Свойство	Описание
<u>closed</u>	логическое значение (true или false) в зависимости от того, закрыто указанное окно или открыто.
<u>length</u>	количество фреймов (включая iframe), которые находятся в данном окне.
<u>name</u>	имя данного окна.
<u>opener</u>	ссылку на окно, которое открыло данное.
<u>parent</u>	родительское окно данного окна.
<u>self</u>	текущее окно.
<u>top</u>	верхнее браузерное окно для данного окна.

Основные методы window

Свойство	Описание
<u>alert()</u>	Вызывает окно оповещения, которое содержит текст сообщения и клавишу ОК
<u>blur()</u>	Делает окно неактивным.
<u>clearInterval()</u>	Прекращает повторное выполнение кода заданного setInterval().
<u>clearTimeout()</u>	Отменяет запланированное методом setTimeout() выполнение кода.
<u>close()</u>	Закрывает окно.
<u>confirm()</u>	Вызывает окно подтверждения содержащее текст сообщения и клавиши ОК и Отмена.
<u>focus()</u>	Делает окно активным.

Основные методы window

Свойство	Описание
<u>moveTo()</u>	Перемещает окно на указанную позицию.
<u>open()</u>	Открывает новое окно браузера. <code>open(url, target, windowFeatures);</code>
<u>print()</u>	Распечатывает содержимое текущего окна.
<u>prompt()</u>	Вызывает окно запроса, побуждающее посетителя ввести в него определенные данные.
<u>scrollBy()</u>	Прокручивает содержимое окна на указанное количество пикселей.
<u>scrollTo()</u>	Прокручивает содержимое окна до указанных координат.
<u>setInterval()</u> <code>clearInterval()</code>	Вызывает функцию или выполняет код через определенные промежутки времени (указанные в миллисекундах). (для остановки таймера)
<code>setTimeout()</code> <code>clearTimeout()</code>	Для однократного выполнения действий через промежуток времени (для остановки таймера)

Демонстрация работы с BOM

```
<!-- Простая страница -->
<!DOCTYPE html>
<html>
<head>
    <title>Window Object</title>
</head>
<body>
    <h1>Window Object</h1>
    <div id="info"></div>
    <!-- Важно место подключения скрипта (в конце body).
         Здесь объектная модель уже полностью сформирована. -->
    <script src="window-object.js"></script>
</body>
</html>
```

Демонстрация работы с BOM

```
/* это файл window-object.js */
var msg = '<h2>browser</h2><p>width: ' + window.innerWidth + '</p>';
msg += '<p>height: ' + window.innerHeight + '</p>';
msg += '<h2>history</h2><p>items: ' + window.history.length + '</p>';

/* window.screen или screen - это то же самое */
msg += '<h2>screen</h2><p>width: ' + screen.width + '</p>';
msg += '<p>height: ' + screen.height + '</p>';

var el = document.getElementById('info');
el.innerHTML = msg;
alert('Current page: ' + window.location);
```

Демонстрация работы с BOM

```
var age = prompt("Введите свой возраст:");  
document.write("Вам " + age + " лет");
```

```
<body>  
<button onclick="openWindow()">Click</button>  
<script>  
function openWindow(){  
  window.open("https://metanit.com", "_blank",  
"width=600,height=400,left=500,top=200, popup=yes ");}  
</script>  
</body>
```

Демонстрация работы с BOM

```
// откроем новое окно и через 5 секунд закроем его  
//функция window.open() возвращает ссылку на окно,  
//и с помощью этой ссылки мы можем управлять окном
```

```
function openWindow(){  
  const popup = window.open("https://metanit.com", "_blank",  
    "width=600,height=400");  
  setTimeout(()=>popup.close(), 5000);} 
```

```
// переместить окно на новую позицию
```

```
const popup = window.open("https://metanit.com", "_blank", "width=600,height=400");  
popup.moveTo(500,400);
```

Демонстрация работы с BOM

//свойство хранит полную строку юзер-агента, например, Mozilla/5.0 (Windows NT 6.3; WOW64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/44.0.2403.157 Safari/537.36

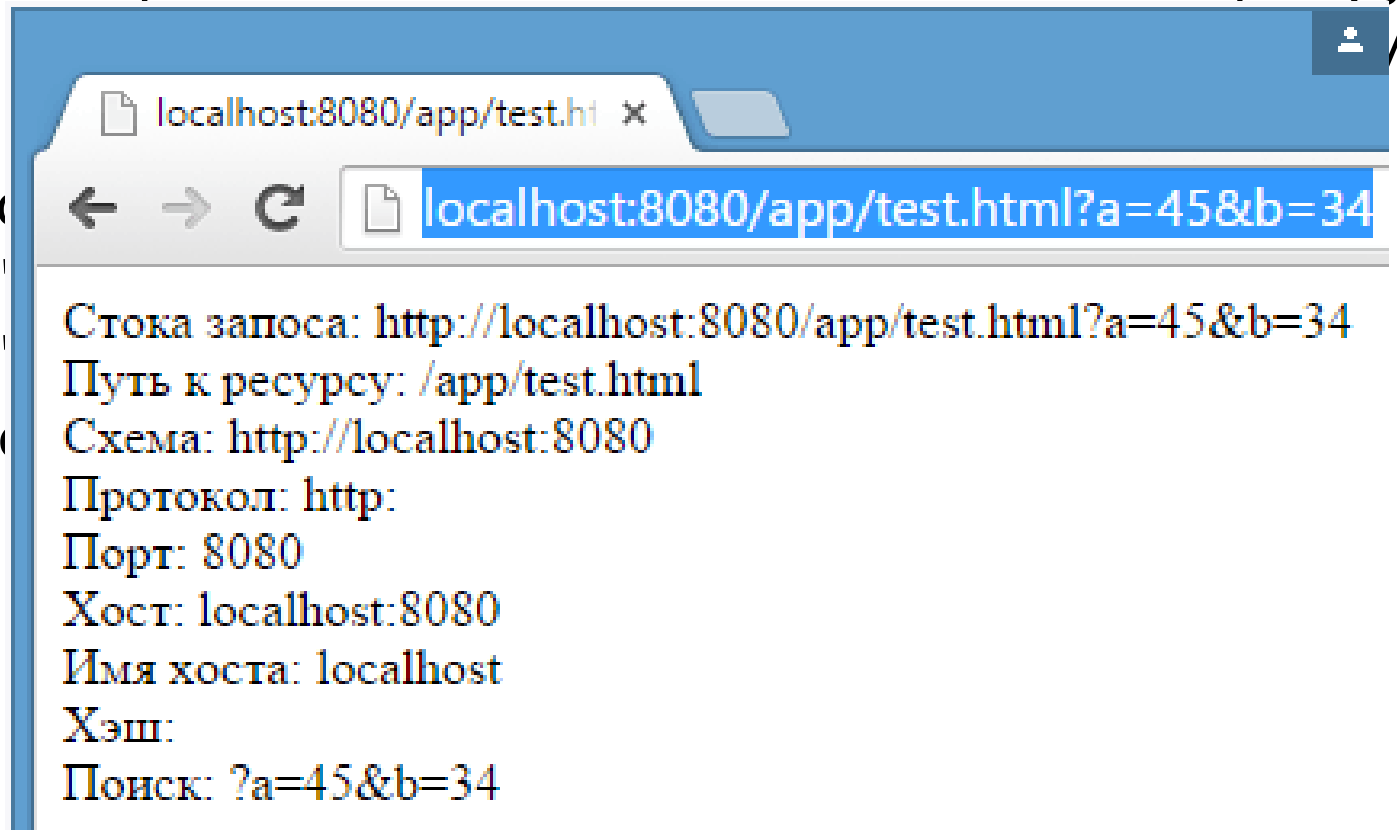
```
document.write(navigator.userAgent);
```

Демонстрация работы с BOM

```
function success(position) {  
    var latitude = position.coords.latitude;  
    var longitude = position.coords.longitude;  
    var altitude = position.coords.altitude;  
    var speed = position.coords.speed;  
    document.write("Широта: " + latitude + "<br/>");  
    document.write("Долгота: " + longitude + "<br/>");  
    document.write("Высота: " + altitude + "<br/>");  
    document.write("Скорость перемещения: " + speed + "<br/>");  
};  
function error(obj) {  
    document.write("Ошибка при определении положения");  
};  
navigator.geolocation.getCurrentPosition(success, error); //географическое положение пользователя
```

Демонстрация работы с BOM

```
document.write("Строка запроса: " + location.href + "<br />");  
document.write("Путь к ресурсу: " + location.pathname + "<br />");  
document.write("Схема: " + location.protocol + "<br />");  
document.write("Протокол: " + location.protocol + "<br />");  
document.write("Порт: " + location.port + "<br />");  
document.write("Хост: " + location.hostname + "<br />");  
document.write("Имя хоста: " + location.hostname + "<br />");  
document.write("Хэш: " + location.hash + "<br />");  
document.write("Поиск: " + location.search + "<br />");
```



Объектная модель документа (DOM)

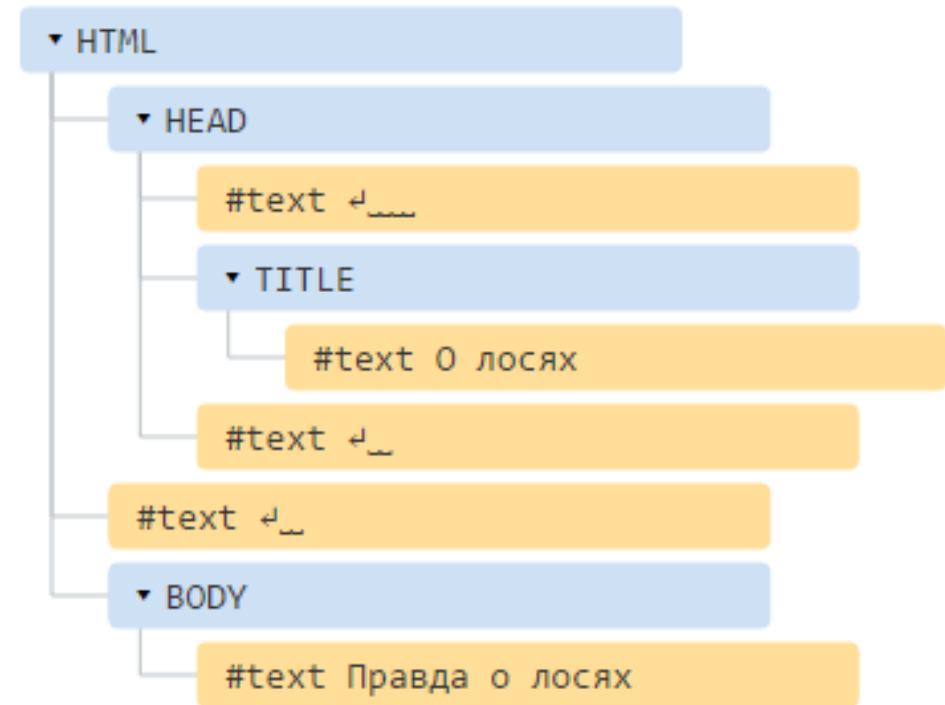
DOM – основной инструмент работы и динамических изменений на странице.

Согласно DOM, документ является иерархией, *деревом*. Каждый HTML-элемент образует узел дерева. Элементы, вложенные в него, становятся дочерними узлами.

Для представления текста создаются узлы особого типа «текст».

Документ и его DOM

```
<!DOCTYPE HTML>
<html>
<head>
  <title>0 лосях</title>
</head>
<body>
  Правда о лосях
</body>
</html>
```



Документ и его DOM

На предыдущем слайде обратите внимание:

- ❑ DOM имеет узлы двух типов. Существуют и другие типы узлов (например, для комментариев).
- ❑ Текстовые узлы созданы для переводов строк и отступов. То, как мы записали исходный html-документ, **влияет на состав DOM.**

Автоисправление DOM

При загрузке нестандартного HTML-документа браузер вносит корректировки при построении DOM:

1. Всегда добавляется элемент `html`.
2. Всегда добавляется элемент `body`.
3. В таблицы добавляется элемент `tbody`.
4. Незакрытые теги автоматически закрываются.

Типовые сценарии работы с DOM

1. Получить элемент

- Выбрать один элемент
- Выбрать несколько элементов
- Перемещаться между элементами

2. Изменить элемент

- Изменить текстовый узел
- Работа с внутренним HTML
- Изменить атрибуты

Навигация — точки входа

Доступ к DOM начинается с объекта `document`.

Первая точка входа — свойство `document.documentElement`. Оно ссылается на DOM-объект для элемента `html`.

Вторая точка входа — свойство `document.body`, которое соответствует элементу `body`.

В современных браузерах (кроме IE8-) есть свойство `document.head` — прямая ссылка на элемент `head`.

Получение элемента по id

Если у элемента установлен атрибут **id**, получить элемент можно по значению этого атрибута:

```
var element = document.getElementById('id_value');
```

Согласно стандарту, значение атрибута **id** должно быть уникально в документе.

Если это не так, то поведение не определено: браузер может вернуть первый или последний из элементов.

Получение элементов по тегу

Метод `elem.getElementsByTagName()` ищет все элементы с заданным тегом внутри элемента `elem` и возвращает их в виде списка.

Регистр тега не имеет значения. Можно получить всех потомков, передав звездочку `'*'` вместо тега.

Внимание: в отличие от `getElementById()`, который существует только в контексте `document`, метод `getElementsByTagName()` может искать внутри **любого элемента**.

Выборка элементов по имени и классу

Метод `document.getElementsByName()` позволяет получить все элементы с указанным значением атрибута **name**. Используется этот метод редко.

Вызов `elem.getElementsByClassName()` возвращает коллекцию элементов с указанным классом. Находит элемент и в том случае, если у него несколько классов, а искомый — один из них.

Получение элементов – пример 1

```
<table id="age-table">
  <tr>
    <td>Ваш возраст:</td>
    <td>
      <input type="radio" class="open" value="young" checked> младше 18
      <input type="radio" value="mature"> от 18 до 50
      <input type="radio" class="open" value="senior"> старше 60
    </td>
  </tr>
</table>
<script>
  // получение элемента по id
  var tableElem = document.getElementById('age-table');
  tableElem.style.background = 'lightgray';
  // получение элементов по тегу и классу
  var elements = tableElem.getElementsByTagName('input');
  var openRange = tableElem.getElementsByClassName('open');
</script>
```

Выборка по CSS-селекторам

Вызов `elem.querySelectorAll()` возвращает все элементы внутри `elem`, удовлетворяющие указанному CSS-селектору:

```
var list = document.querySelectorAll('ul > li');
```

Метод `elem.querySelector()` возвращает не все, а только первый элемент, соответствующий селектору.

Выборка по CSS-селекторам

Метод `elem.closest()` ищет ближайший элемент выше по иерархии DOM, подходящий под CSS-селектор. Сам элемент тоже включается в поиск.

Иначе говоря, `closest()` бежит от текущего элемента вверх по цепочке родителей и проверяет, подходит ли элемент под указанный селектор. Если подходит – останавливается и возвращает его.

Поддержка браузеров: [http://caniuse.com/#search=closest\(\)](http://caniuse.com/#search=closest())

Тест элемента CSS-селектором

Метод `elem.matches()` просто проверяет, удовлетворяет ли элемент указанному селектору, и возвращает `true` или `false`. Этот метод полезен при фильтрации.

Ранее в спецификации этот метод назывался `matchesSelector()`, и большинство браузеров поддерживают его под этим старым именем.

Получение элементов – сводка

Метод	Ищет по...	Ищет внутри элемента?	Возвращаемая коллекция
<code>getElementById()</code>	id	нет	–
<code>getElementsByName()</code>	name	нет	«Живая»
<code>getElementsByTagName()</code>	тег или '*'	да	«Живая»
<code>getElementsByClassName()</code>	класс	да	«Живая»
<code>querySelector()</code>	CSS-селектор	да	Статическая
<code>querySelectorAll()</code>	CSS-селектор	да	Статическая
<code>closest()</code>	CSS-селектор	да	–

Навигация по DOM-элементам

Коллекция `elem.childNodes` хранит все дочерние элементы DOM (включая текстовые). Свойства `elem.firstChild` / `elem.lastChild` ссылаются на первый/последний дочерний элемент.

Доступ к элементам DOM слева и справа данного можно получить по ссылкам `elem.previousSibling` / `elem.nextSibling`.

Родитель доступен через `elem.parentNode`.

Навигация по элементам

Если нужна навигация не по элементам дерева DOM, а именно по элементам HTML из документа:

- `children` – дочерние элементы.
- `firstElementChild`, `lastElementChild` – соответственно, первый и последний дети-элементы.
- `previousElementSibling`, `nextElementSibling` – соседи-элементы.
- `parentElement` – родитель-элемент.

Отступление — о коллекциях

Упомянутые методы, которые возвращают коллекцию, возвращают объект `NodeList`.

Это объект, подобный массиву. Однако, при доступе по индексу и отсутствии элемента возвращает `null`.

Если браузер поддерживает `for-of`, то объект `NodeList` можно с помощью этого цикла перечислить.

Навигация по таблице

У конкретных элементов DOM могут быть дополнительные свойства для удобства навигации.

В частности, у таблицы и её дочерних элементов:

table

- `table.rows` – коллекция строк **tr** таблицы.
- `table.caption/thead/tfoot` – ссылки на элементы таблицы **caption**, **thead**, **tfoot**.
- `table.tbodies` – коллекция элементов таблицы **tbody**, по спецификации их может быть несколько.

Навигация по таблице

thead/tfoot/tbody

- `tbody.rows` – коллекция строк **tr**

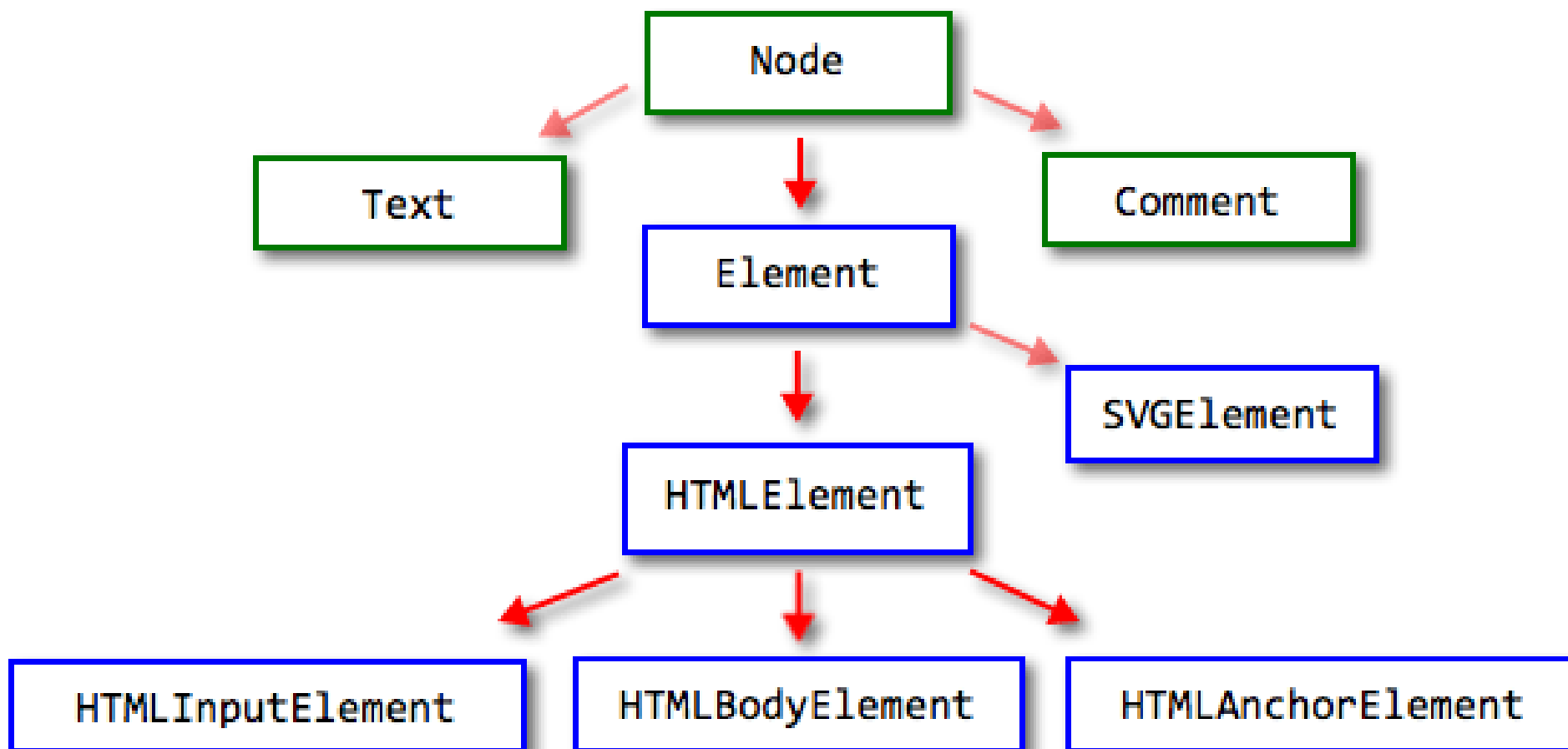
tr

- `tr.cells` – коллекция ячеек **td/th**
- `tr.sectionRowIndex` – номер строки в текущей секции **thead/tbody**
- `tr.rowIndex` – номер строки в таблице

td/th

- `td.cellIndex` – номер ячейки в строке

Иерархия классов элементов DOM



Иерархия классов элементов DOM

- `Node` – базовый класс.
- Прямо от `Node` наследуются текстовые узлы `Text`, комментарии `Comment` и элементы `Element`.
- `Element` – общий тип, используемый в SVG и HTML. От него наследуются `SVGElement` для SVG-графики и `HTMLElement`.
- От `HTMLElement` наследуют разнообразные узлы HTML (`input` – `HTMLInputElement`, `body` – `HTMLBodyElement`, `a` – `HTMLAnchorElement`, и так далее).

Свойства элементов DOM

Тип узла содержится в его свойстве `nodeType`.

```
interface Node {  
    // Всевозможные значения nodeType  
    const unsigned short ELEMENT_NODE = 1;  
    const unsigned short ATTRIBUTE_NODE = 2;  
    const unsigned short TEXT_NODE = 3;  
    const unsigned short CDATA_SECTION_NODE = 4;  
    const unsigned short ENTITY_REFERENCE_NODE = 5;  
    const unsigned short ENTITY_NODE = 6;  
    const unsigned short PROCESSING_INSTRUCTION_NODE = 7;  
    const unsigned short COMMENT_NODE = 8;  
    const unsigned short DOCUMENT_NODE = 9;  
    const unsigned short DOCUMENT_TYPE_NODE = 10;  
    const unsigned short DOCUMENT_FRAGMENT_NODE = 11;  
    const unsigned short NOTATION_NODE = 12;  
    ...  
}
```

Свойства элементов DOM

Свойство `tagName` – это имя элемента, оно есть только у `Element` (в IE8- также у комментариев, но это ошибка в браузере).

Свойство `nodeName` определено для любых узлов `Node`, для элементов оно равно `tagName`, а для не-элементов обычно содержит строку с типом узла.

Свойства элементов DOM

```
<script>
    // все дочерние (НЕ ПОТОМКИ) у body
    var childNodes = document.body.childNodes;
    for (var i = 0; i < childNodes.length; i++) {
        // выводим тип
        alert(childNodes[i].nodeType);

        // фильтруем элементы
        if (childNodes[i].nodeType === 1) {
            // у элементов выводим тег
            alert(childNodes[i].tagName);
        }
    }
</script>
```

Свойства элементов DOM

Свойство `innerHTML` позволяет получить HTML-содержимое элемента в виде строки. Это свойство доступно и для чтения, **и для записи**.

Свойство `outerHTML` содержит HTML элемента целиком. Технически свойство `outerHTML` доступно на запись. Но при этом элемент не меняется, а заменяется на новый, который тут же создаётся из нового `outerHTML`. Поэтому будем считать, что изменить `outerHTML` нельзя.

Свойства элементов DOM

```
<body>
  <p>Параграф</p>
  <div>Div</div>

  <script>
    alert(document.body.innerHTML); // читаем содержимое
    document.body.innerHTML = 'Новый BODY!'; // заменяем

    // браузер исправит ошибки:
    document.body.innerHTML = '<b>тест'; // незакрытый тег
    alert(document.body.innerHTML); // <b>тест</b> (исправлено)
  </script>
</body>
```

Свойства элементов DOM

Свойство `innerHTML` есть только у `Element`.

Содержимое других узлов, например, текстовых или комментариев, доступно на чтение и запись через свойство `data`.

Его тоже можно читать и обновлять.

Свойства элементов DOM

Свойство `textContent` содержит только текст внутри элемента, за вычетом всех **<тегов>**. Иными словами, `textContent` возвращает конкатенацию всех текстовых узлов внутри элемента.

При записи в `textContent` строка учитывается как текст, без тегов (а не как фрагмент HTML – и это отличие от `innerHTML`).

Свойства элементов DOM

```
<div></div>
```

```
<div></div>
```

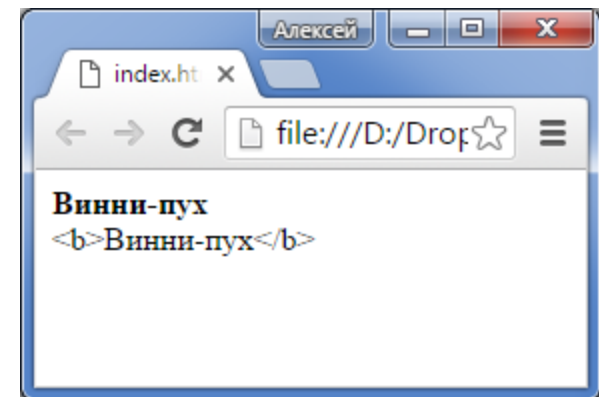
```
<script>
```

```
    var name = prompt("Введите имя?", "<b>Винни-пух</b>");
```

```
    document.body.children[0].innerHTML = name;
```

```
    document.body.children[1].textContent = name;
```

```
</script>
```



Свойства элементов DOM

Как исследовать свойства элементов DOM?

1. <http://w3c.github.io/html-reference/elements.html>
2. Выводим элемент в консоль вызовом `console.dir(элемент)`. Метод `console.dir()` выводит аргумент как объект, который можно развернуть и исследовать.

Атрибуты и DOM-свойства

Если узел DOM наследуется от `Element`, он имеет методы для работы с атрибутами:

- `elem.hasAttribute(name)` – проверяет наличие атрибута
- `elem.getAttribute(name)` – получает значение атрибута
- `elem.setAttribute(name, value)` – устанавливает атрибут
- `elem.removeAttribute(name)` – удаляет атрибут

Все атрибуты можно получить с помощью свойства `elem.attributes` – коллекция объектов типа `Attr`.

Атрибуты и DOM-свойства

```
<body>
  <div id="elem" about="Elephant"></div>
<script>
  var elem = document.getElementById("elem");
  // имена атрибутов не чувствительны к регистру
  alert(elem.getAttribute("About"));
  // атрибут Test установлен
  elem.setAttribute("Test", 123);
  alert(document.body.innerHTML);
  // работаем с коллекцией атрибутов
  var attrs = elem.attributes;
  for (var i = 0; i < attrs.length; i++) {
    alert(attrs[i].name + " = " + attrs[i].value);
  }
</script>
</body>
```

Атрибуты и DOM-свойства

Когда браузер читает HTML и создаёт DOM, он создаёт свойства у элементов для всех **стандартных** атрибутов.

Например, у элемента **a** есть стандартный атрибут **href**.
Значит, у объекта в DOM для **a** будет свойство href.

```
<body>
  <a id="link" href="http://issoft.by"></a>
<script>
  var link = document.getElementById("link");
  alert(link.href);
  link.href = "http://mail.ru";
  alert(document.body.innerHTML);
</script>
</body>
```

Атрибуты и DOM-свойства

Значение атрибута не всегда соответствует значению свойства.

1. Свойство `input.checked` имеет значение `boolean`, а HTML-атрибут `checked` может быть любой строкой.
2. Свойство `href` будет полной ссылкой, а HTML-атрибут `href` может быть любой строкой.
3. Изменение свойства `value` не меняет значение атрибута `value`.

Атрибуты и DOM-свойства

Так как `class` – зарезервированное слово в JavaScript, атрибуту `class` соответствует свойство `className`.

Свойство `classList` – это *объект* для работы с классами (удобно, так как классов может быть больше одного).

Атрибуты и DOM-свойства

В HTML5 описаны специальные **data**-атрибуты. При этом во всех браузерах (кроме IE10-) к таким атрибутам можно обратиться как к свойствам, при помощи специального свойства `dataset`:

```
<div id="elem" data-about="Elephant" data-user-location="street">
```

По улице прошёлся слон. Весьма красив и толст был он.

```
</div>
```

```
<script>
```

```
var elem = document.getElementById("elem");  
alert(elem.dataset.about); // Elephant +  
// user-location превратилось в userLocation  
alert(elem.dataset.userLocation); // street
```

```
</script>
```

Создание узлов

Для создания узлов используются следующие методы:

`document.createElement(tag)` – создает новый элемент с указанным тегом

`document.createTextNode(text)` – создает новый текстовый узел с данным текстом

Добавление узлов

Чтобы DOM-узел был показан на странице, его необходимо вставить в `document`. Пусть мы будем вставлять узел в некий элемент `parentElem`.

`parentElem.appendChild(elem)` – добавляет `elem` в конец дочерних элементов `parentElem`.

`parentElem.insertBefore(elem, nextSibling)` – вставляет `elem` в коллекцию детей `parentElem`, перед элементом `nextSibling`.

Добавление узлов – пример

```
<ol id="list">
  <li>0</li>
  <li>1</li>
  <li>2</li>
</ol>
<script>
  var newLi = document.createElement('li');
  newLi.innerHTML = 'Привет, мир!';

  var list = document.getElementById("list");
  list.appendChild(newLi);
</script>
```

Клонирование узлов

Вызов `elem.cloneNode(true)` создаст «глубокую» копию элемента – вместе с атрибутами, включая подэлементы.

Если же вызвать с аргументом `false`, то копия будет сделана без дочерних элементов. Это нужно гораздо реже.

Полученный клон можно немного изменить и вставить в документ.

Добавление узлов

Недавно в стандарте DOM появились методы, которые позволяют вставить что угодно и куда угодно:

- `node.append(...nodes)` – вставляет `nodes` в конец `node`
- `node.prepend(...nodes)` – вставляет `nodes` в начало `node`
- `node.after(...nodes)` – вставляет `nodes` после узла `node`
- `node.before(...nodes)` – вставляет `nodes` перед узлом `node`
- `node.replaceWith(...nodes)` – вставляет `nodes` вместо `node`

Эти методы ничего не возвращают. `nodes` – DOM-узлы или строки. Строки вставляются как текстовые узлы.

Удаление узлов

`parentElem.removeChild(elem)` – удаляет `elem` из списка детей `parentElem`.

`parentElem.replaceChild(newElem, elem)` – среди детей `parentElem` удаляет `elem` и вставляет на его место `newElem`.

Оба метода возвращают удаленный узел, то есть `elem`. Если нужно, его можно вставить в другое место DOM тут же или в будущем.

Удаление узлов

В современном стандарте есть также метод `elem.remove()`, который удаляет элемент напрямую, не требуя ссылки на родителя. Это зачастую удобнее, чем `removeChild()`.

Метод `elem.remove()` поддерживается во всех современных браузерах, кроме IE11-.

Метод `document.write()`

Метод `document.write(str)` работает только пока HTML-страница находится в процессе загрузки. Он дописывает текст в текущее место HTML ещё до того, как браузер построит из него DOM.

Нет никаких ограничений на аргумент метода! Строка просто пишется в HTML-документ без проверки структуры тегов, как будто она всегда там была.

Метод document.write() – пример

```
<script>
    document.write('<style> td { color: #F40 } </style>');
</script>
<table>
    <tr>
        <script>
            document.write('<td>')
        </script>
        Текст внутри TD.
        <script>
            document.write('</td>')
        </script>
    </tr>
</table>
```

Работа со стилями

Свойство `element.style` возвращает **объект**, который даёт доступ к стилю элемента на чтение и запись. С его помощью можно изменять большинство CSS-свойств.

Например `element.style.width="100px"` работает так, как будто у элемента в атрибуте прописано `style="width:100px"`.

Работа со стилями — нюансы

Для значений свойств нужно указывать единицы измерения (например px).

Для CSS-свойств, названия которых состоят из нескольких слов, используется вотТакаяЗапись:
`background-color` => `elem.style.backgroundColor`

Свойства браузеров типа `-moz-border-radius` записываются так: `button.style.MozBorderRadius`

Чтобы сбросить свойство, присвойте пустую строку

Работа со стилями

Свойство `element.cssText` позволяет прочитать или задать CSS в виде строки.

При установке этого свойства все предыдущие стили удаляются.

```
div.style.cssText=`color: red !important;
background-color: yellow;
width: 100px;
text-align: center;
`;
```

Работа со стилями

При чтении CSS-свойств из `element.style` мы получаем только те свойства, что были применены непосредственно к элементу.

Метод `window.getComputedStyle(element[, pseudo])` возвращает **вычисленный стиль**:

```
var computed = getComputedStyle(document.body);  
alert(computed.marginTop);
```